



第14课时：二维数组

定义 · 访问 · 遍历 · 矩阵操作

二维数组的定义与访问、双重循环遍历、矩阵的简单操作



为什么需要二维数组？



一维数组可以存储"一行"数据，但现实中很多数据是**二维**的：

- **成绩表**：30名学生 × 5门科目
- **棋盘**：8行 × 8列
- **图像**：像素矩阵（行 × 列）
- **地图**：网格坐标

如果用一维数组表示，需要手动计算下标，代码可读性差。**二维数组**正是为这类问题而生——它用"行"和"列"两个下标来管理数据。

二维数组的概念



二维数组是具有**两个维度**的数组，可以看作"数组的数组"。

- 第一个下标表示**行** (row)
- 第二个下标表示**列** (column)
- 行下标和列下标都**从 0 开始**

💡 **生活类比**：二维数组就像一张**Excel表格**或**电影院座位表**

- 行 = 表格的行
- 列 = 表格的列

- `arr[2][3]` = 第3行第4列的元素（从0开始数）

二维数组的声明



声明二维数组需要指定：**类型、数组名、行数、列数**。

```
// 语法
类型 数组名[行数][列数];

// 示例
int scores[30][5];      // 30行5列的成绩表
char board[8][8];       // 8×8 棋盘
double matrix[3][3];    // 3×3 矩阵
```

⚠ 注意：

- 行数和列数都必须是**编译期常量**
- 总元素个数 = 行数 × 列数
- 内存中仍然是**连续存储**（按行优先排列）



二维数组的初始化



可以在声明时直接初始化二维数组。

方式一：完全初始化

```
int arr[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

方式二：部分初始化（未指定元素自动为0）

```
int arr[2][3] = {  
    {1, 2},  
    {4}  
};
```

方式三：全部初始化为0

```
int arr[3][3] = {};    // 所有元素为 0  
// 或  
int arr[3][3] = {0};
```

二维数组的访问



通过 数组名[行下标][列下标]
访问或修改元素。

来访

```
int arr[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

⚠ 重要:

- 行下标范围:

0

行数-1

~

- 列下标范围:

0

列数-1

~

- 越界访问会导致**未定义行为**

二维数组的内存布局



二维数组在内存中是**按行优先**连续存放的。

例如 `int arr[2][3]`

内存布局：

```
arr[0][0] arr[0][1] arr[0][2] arr[1][0] arr[1][1] arr[1][2]
  1         2         3         4         5         6
```

- 先存储第0行的所有元素，再存储第1行的所有元素
- 每个元素占用的字节数由数据类型决定（

`int`

节)

- 总内存 = 行数 × 列数 × sizeof(元素类型)

占 4 字

理解这个特性有助于后续学习指针与数组的关系。

二维数组的遍历（双重循环）



遍历二维数组需要使用**双重循环**：外层循环控制行，内层循环控制列。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int arr[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    // 双重循环遍历
    for (int i = 0; i < 2; i++) {           // 外层: 行
        for (int j = 0; j < 3; j++) {       // 内层: 列
            cout << arr[i][j] << " ";
        }
        cout << endl;                     // 每行结束换行
    }
    return 0;
}
// 输出:
// 1 2 3
// 4 5 6
```

双重循环的执行过程



以 `arr[2][3]` 为例，
双重循环的执行顺序：

外层 i	内层 j	访问元素	输出
0	0	<code>arr[0][0]</code>	1
0	1	<code>arr[0][1]</code>	2
0	2	<code>arr[0][2]</code>	3
1	0	<code>arr[1][0]</code>	4
1	1	<code>arr[1][1]</code>	5
1	2	<code>arr[1][2]</code>	6

例题1：二维数组的输入与输出



题目：输入一个 $n \times m$ 的矩阵，按原样输出。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    int arr[100][100];

    // 输入
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> arr[i][j];
        }
    }

    // 输出
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cout << arr[i][j];
            if (j < m - 1) cout << " ";
        }
        cout << endl;
    }
    return 0;
}
```

例题2：求矩阵每行的和



题目：输入一个 $n \times m$ 的矩阵，求每行元素的和。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    int arr[100][100];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cin >> arr[i][j];

    // 求每行的和
    for (int i = 0; i < n; i++) {
        int rowSum = 0;
        for (int j = 0; j < m; j++) {
            rowSum += arr[i][j];
        }
        cout << "第" << i + 1 << "行的和: " << rowSum << endl;
    }
    return 0;
}
```

例题3：求矩阵的最大值及其位置



题目：输入一个 $n \times m$ 的矩阵，找出最大值及其所在的行和列。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    int arr[100][100];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cin >> arr[i][j];

    int maxVal = arr[0][0];
    int maxRow = 0, maxCol = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (arr[i][j] > maxVal) {
                maxVal = arr[i][j];
                maxRow = i;
                maxCol = j;
            }
        }
    }

    cout << "最大值: " << maxVal << ", 位置: 第" << maxRow + 1
         << "行第" << maxCol + 1 << "列" << endl;
```

矩阵的转置



转置：将矩阵的行和列互换，即

```
result[j][i] = arr[i][j]
```

对于一个 $n \times m$ 的矩阵，转置后变为 $m \times n$ 的矩阵。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int arr[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    int result[3][2]; // 转置后为 3x2

    // 转置
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            result[j][i] = arr[i][j];
        }
    }

    // 输出转置后的矩阵
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
```

矩阵的主对角线



主对角线：从左上到右下的对角线，元素满足

行下标 == 列下标

$i == j$

(即
)

对于一个 $n \times n$ 的方阵：

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int arr[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // 输出主对角线元素
    cout << "主对角线: ";
    for (int i = 0; i < 3; i++) {
        cout << arr[i][i] << " "; // i == j
    }
    cout << endl;
    // 输出: 主对角线: 1 5 9
```

矩阵的副对角线



副对角线：从右上到左下的对角线，元素满足

$$\text{行下标} + \text{列下标} == n - 1$$

(即

$$i + j == n - 1$$

)。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int arr[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    int n = 3;

    // 输出副对角线元素
    cout << "副对角线: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i][n - 1 - i] << " "; // i + j == n - 1
    }
    cout << endl;
    // 输出: 副对角线: 3 5 7

    // 求副对角线元素之和
    int sum = 0;
    for (int i = 0; i < n; i++) {
```

矩阵的边界输出



题目：输出一个 $n \times n$ 矩阵的边界元素（第一行、最后一行、第一列、最后一列）。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    int arr[100][100];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> arr[i][j];

    // 输出边界元素
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == 0 || i == n - 1 || j == 0 || j == n - 1) {
                cout << arr[i][j] << " ";
            } else {
                cout << " "; // 非边界元素用空格代替
            }
        }
    }
}
```

💡 关键条件：

$i == 0 \ || \ i == n-1 \ || \ j == 0 \ || \ j == n-1$

矩阵加法



矩阵加法：两个同型矩阵（行数和列数相同）对应位置的元素相加。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int A[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    int B[2][3] = {
        {7, 8, 9},
        {10, 11, 12}
    };
    int C[2][3]; // 结果矩阵

    // 矩阵加法: C = A + B
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    // 输出结果
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cout << C[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

常见错误与调试



错误类型	示例/说明	解决方法
下标越界	arr[n][m] 当数组大小为 n×m 时越界	确保下标范围 0~行数-1 和 0~列数-1
行列搞反	外层循环用列、内层用行	记住"外行内列": 外层 i 控 制行, 内层 j 控 制列
未初始化	局部二维数组未初始化, 元素为随机值	声明时用 {} 或 {0} 初 始化

二维数组的典型应用



应用场景	说明
成绩管理	存储多名学生的多科成绩
棋盘游戏	围棋、五子棋、国际象棋的棋盘
图像处理	像素矩阵的存储与变换
地图建模	网格地图、迷宫表示
动态规划	状态转移表（后续学习）
邻接矩阵	图的存储（后续学习）

二维数组是处理**表格型数据**和**矩阵运算**的核心数据结构。



知识小结



✓ 二维数组的定义：

类型 数组名[行数][列数];

，行和列都

从 0 开始

✓ 二维数组的访问：

arr[行][列]

，注意下标

范围

✓ 初始化：用嵌套大括号

{{}}, {} }

初始化，未

指定的自动为 0

✓ 双重循环遍历：外层控制行，内层控制列（"外行内列"）

✓ 矩阵操作：转置、对角线、边界、加法等

✓ 内存布局：按行优先连续存储



下节课预告



下一阶段：函数与结构化编程

- 函数的定义与调用
- 参数传递与返回值
- 局部变量与全局变量
- 递归入门

"从一维到二维，是思维的一次飞跃。掌握多维数据，才能应对更复杂的世界。"

谢谢大家！



二维数组打开了多维数据处理的大门！

Q & A

